Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 5

Recurrences (Part 2)

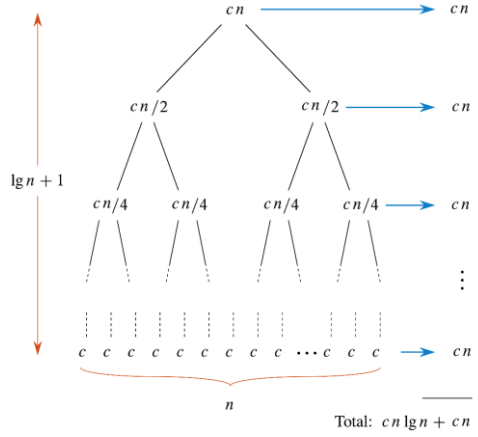
Αναδρομικές Σχέσεις (Μέρος 2)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

The recursion tree method

- We already saw this method in Merge-Sort
  - $T(n) = 2T(n/2) + cn$

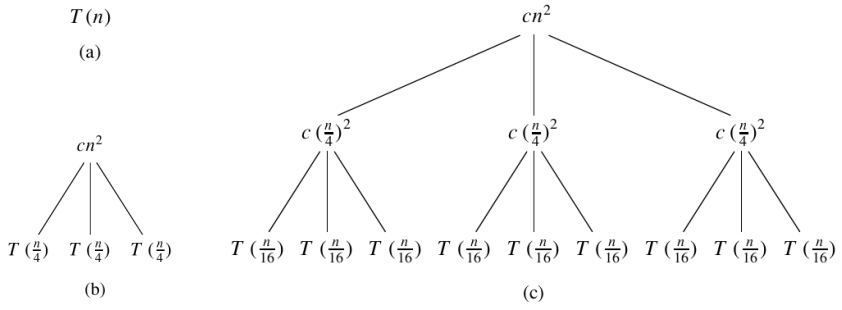


The recursion tree method

- **Recursion tree** (**Δένδρο αναδρομής**)
  - Each node represents the cost of a single subproblem
  - Sum the costs at each level to obtain a per-level cost
  - Sum the per-level costs to obtain the total cost
- It is a good method for producing a guess, which we can then prove with the **substitution method** (**μέθοδος της αντικατάστασης**)
  - A small amount of "sloppiness" ("προχειρότητα") can be tolerated, since we will verify it later on
- If we are very careful when drawing out a recursion tree and summing the costs, we can use a recursion tree as a direct proof!

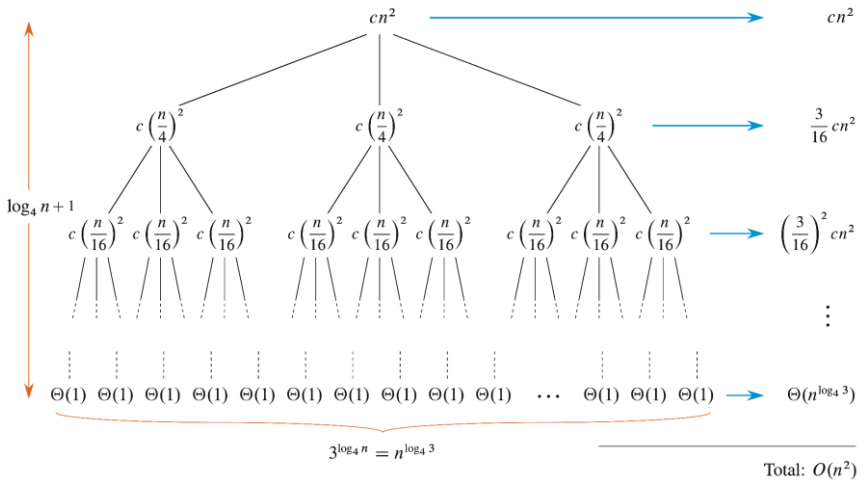
Example

- Let us find a good guess for $\mathrm{T}(n) = 3\mathrm{T}(\lfloor n/4 \rfloor) + \Theta(n^2)$
- Sloppiness: ignore the floor function $\mathrm{T}(n) = 3\mathrm{T}(n/4) + cn^2$
- More sloppiness: assume that n is a power of 4



- Continue expanding, until you reach a boundary condition

Example (cont'd)



Example (cont'd)

- How many levels does the tree have?
  - Problem size at depth i (start counting from 0): $n/4^i$
  - At the bottom: $n/4^i = 1 \Rightarrow n = 4^i \Rightarrow i = \log_4 n$
  - Number of levels: $1 + \log_4 n$
- How many leafs does the tree have?
  - At depth i (start counting from 0) there are 3^i nodes
  - At the last level ($i = \log_4 n$), there are $3^{\log_4 n} = n^{\log_4 3}$ nodes

Example (cont'd)

- What is the cost at each level?
  - Each node has cost $c(n/4^i)^2$
  - At depth i the cost is $3^i c(n/4^i)^2 = (3/16)^i cn^2$
  - The last level, there are $n^{\log_4 3}$ nodes, each contributing constant cost, $n^{\log_4 3} T(1) = \Theta(n^{\log_4 3})$

- Add up the costs of the entire tree:

$$\begin{aligned} T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\ &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3}) . \end{aligned}$$

Geometric Series:
$$\sum_{i=0}^{n} x^i = \frac{x^{n+1} - 1}{x - 1}$$

Example (cont'd)

- Recall, we want to *bound* $T(n)$ and *not* necessarily find its exact value
- Note that the geometric series in front of cn^2 is decreasing
- **Idea:** bound $T(n)$ using the infinite series $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}, |x| < 1$

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\ &= O(n^2) . \end{aligned}$$

- Notice (on the tree) that already at the first iteration the cost is $cn^2 = \Theta(n^2)$
- So, a lower bound for the recurrence must be $\Omega(n^2)$
- So, what we found is actually a tight bound, $\boldsymbol{\Theta(n^2)}$

Example (cont'd)

- Now, we must *verify* our guess with the *substitution method*
- Recall: $\mathrm{T}(n) = 3\mathrm{T}(\lfloor n/4 \rfloor) + \Theta(n^2)$
- We must show that $T(n) \leq dn^2$ for some $d > 0$
- Using the same constant c as before

$$\begin{aligned} T(n) &\leq 3T(\lfloor n/4 \rfloor) + cn^2 \\ &\leq 3\, d\, (n/4)^2 + cn^2 \\ &= \tfrac{3}{16}\, d\, n^2 + cn^2 \\ &= d\, n^2 - \tfrac{13}{16}\, d\, n^2 + cn^2 \\ &\leq d\, n^2, \text{ as long as } -\tfrac{13}{16} dn^2 + cn^2 \leq 0 \Rightarrow d \geq \tfrac{16}{13} c \end{aligned}$$

Exercise

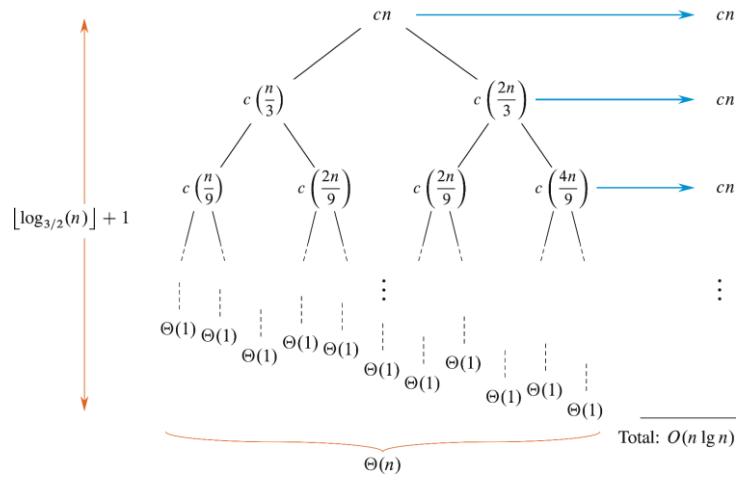
1. Using a recursion tree, obtain a guess for an upper bound of the following recursive equation

$$T(n) = T(n/3) + T(2n/3) + cn$$

Then, prove your guess using the substitution method (floor and ceiling functions are omitted for simplicity)

Solution

$$T(n) = T(n/3) + T(2n/3) + cn$$



Solution

$$T(n) = T(n/3) + T(2n/3) + cn$$

- Guess: $T(n) \leq dn \lg n$ for some $d > 0$
- Substitution:

$$\begin{aligned} T(n) &= T(n/3) + T(2n/3) + cn \\ &\leq d(n/3)\lg(n/3) + d(2n/3)\lg(2n/3) + cn \\ &= d(n/3)\lg(n) - d(n/3)\lg 3 \\ &\quad + d(2n/3)\lg 2 + d(2n/3)\lg n - d(2n/3)\lg 3 + cn \\ &= dn\lg n - dn\lg 3 + d(2n/3) + cn \\ &= dn\lg n - dn(\lg 3 - 2/3) + cn \\ &\leq dn\lg n \end{aligned}$$

as long as $-dn(\lg 3 - 2/3) + cn \leq 0 \Rightarrow d \geq \frac{c}{\lg 3 - 2/3}$

The master method

- Provides solution to recurrences of the form
$$T(n) = aT(n/b) + f(n)$$
where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an **asymptotically positive** (**ασυμπτωτικά θετική**) function
- Requires memorization of three cases
  - but you can then solve many recurrences easily
- Note that n/b might not be an integer! But adding floors and ceilings does not affect asymptotic behaviour, so we omit them for convenience
- The method depends on the **master theorem** (**κεντρική μέθοδος**)

The master theorem

- Let $a \geq 1$ and $b > 1$, and $T(n)$ be defined on non-negative integers by the recurrence
$$T(n) = aT(n/b) + f(n)$$
where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$
Then $T(n)$ can be bounded asymptotically as follows:
  1. If $f(n) = O(n^{(\log_b a)-\epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$
  2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$
  3. If $f(n) = \Omega(n^{(\log_b a)+\epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant c < 1 and all sufficiently large n, then $T(n) = \Theta(f(n))$

The master theorem: Discussion

- Notice, we are comparing $f(n)$ with $n^{\log_b a}$
  - If $n^{\log_b a}$ is polynomially larger (the first case), then
    $$T(n) = \Theta(n^{\log_b a})$$
  - If $f(n)$ is polynomially larger (the third case), then
    $$T(n) = \Theta(f(n))$$
  - If the two functions are of the same size (the second case), then we multiply by a logarithmic factor
    $$T(n) = \Theta\left(n^{\log_b a} \lg n\right) = \Theta(f(n) \lg n)$$

- In all other cases, the master theorem cannot be applied!
  - E.g., if $f(n)$ is smaller or larger than $n^{\log_b a}$ but not polynomially
  - E.g., if the condition $af(n/b) \leq cf(n)$ in case 3 does not hold

Examples

- Consider $T(n) = 9T(n/3) + n$
  - $a = 9, b = 3, f(n) = n$ and $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$
  - Since $f(n) = O\left(n^{(\log_3 9) - \varepsilon}\right)$, where $\varepsilon = 1$, we have **case 1**
  - Therefore, $T(n) = \Theta(n^2)$

- Consider $T(n) = T(2n/3) + 1$
  - $a = 1, b = 3/2, f(n) = 1$ and $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$
  - Since $f(n) = \Theta\left(n^{\log_b a}\right) = \Theta(1)$, we have **case 2**
  - Therefore, $T(n) = \Theta(\lg n)$

Examples (cont'd)

- Consider $T(n) = 3T(n/4) + n \lg n$
  - $a = 3, b = 4, f(n) = n \lg n$ and $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$
  - Since $f(n) = \Omega\left(n^{(\log_4 3)+\varepsilon}\right)$, where $\varepsilon \approx 0.2$, **case 3 might** apply
  - For sufficiently large n,
    $af(n/b) = 3(n/4)\lg(n/4) \leq (3/4)\, n \lg n = cf(n)$
    for $c = 3/4$
  - Therefore, $T(n) = \Theta(n \lg n)$

Examples (cont'd)

- Consider $T(n) = 2T(n/2) + n \lg n$
  - $a = 2, b = 2, f(n) = n \lg n$ and $n^{\log_b a} = n^{\log_2 2} = n$
  - It seems case 3 applies, because $f(n) = n \lg n$ is asymptotically larger than $n^{\log_b a} = n$
  - BUT, it is not polynomially larger!
  - The ratio $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ is asymptotically less than n^{ε} for any positive constant ε
  - So, case 3 **cannot** be applied!

Exercises

1. Use the master theorem to give tight asymptotic bounds for the following recurrences

$$T(n) = 4T(n/2) + n$$
$$T(n) = 4T(n/2) + n^2$$
$$T(n) = 4T(n/2) + n^3$$

2. Use the master theorem to show that the solution to the binary-search recurrence $T(n) = T(n/2) + \Theta(1)$ is $T(n) = \Theta(\lg n)$

Solutions

1. a) $T(n) = 4T(n/2) + n$
   - $a = 4, b = 2, f(n) = n$ and $n^{\log_b a} = n^{\log_2 4} = \Theta(n^2)$
   - Since $f(n) = n = O(n^{2-\varepsilon})$, where $\varepsilon = 1$, we have **case 1**
   - Therefore, $T(n) = \Theta(n^2)$

1. b) $T(n) = 4T(n/2) + n^2$
   - $a = 4, b = 2, f(n) = n$ and $n^{\log_b a} = n^{\log_2 4} = \Theta(n^2)$
   - Since $f(n) = n^2 = \Theta\left(n^{\log_b a}\right) = \Theta(\mathrm{n}^2)$, we have **case 2**
   - Therefore, $T(n) = \Theta(n^2 \lg n)$

Solutions

1. c) $T(n) = 4T(n/2) + n^3$
   - $a = 4, b = 2, f(n) = n$ and $n^{\log_b a} = n^{\log_2 4} = \Theta(n^2)$
   - Since $f(n) = n^3 = \Omega(n^{2+\varepsilon})$, where $\varepsilon = 1$, **case 3 might** apply
   - For sufficiently large n,
     $af(n/b) = 4(n/2)^3 = (1/2)\, n^3 \leq cf(n)$ for $c \geq 1/2$
   - Therefore, $T(n) = \Theta(n^3)$

2) Consider $T(n) = T(n/2) + \Theta(1)$
   - $a = 1, b = 2, f(n) = \Theta(1)$ and $n^{\log_b a} = n^0 = \Theta(1)$
   - Since $f(n) = \Theta(1) = \Theta\left(n^{\log_b a}\right)$, we have **case 2**
   - Therefore, $T(n) = \Theta(\lg n)$

Refresh some basic maths

- Arithmetic series

$$\sum_{k=1}^{n} k = 1 + 2 + \cdots + n = \frac{1}{2} n(n+1)$$

- Geometric series

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

- Infinite geometric series

$$\sum_{k=0}^{\infty} x^k = 1 + x + x^2 + \cdots = \frac{1}{1 - x}$$

- Have a look at CLRS Appendix A for more